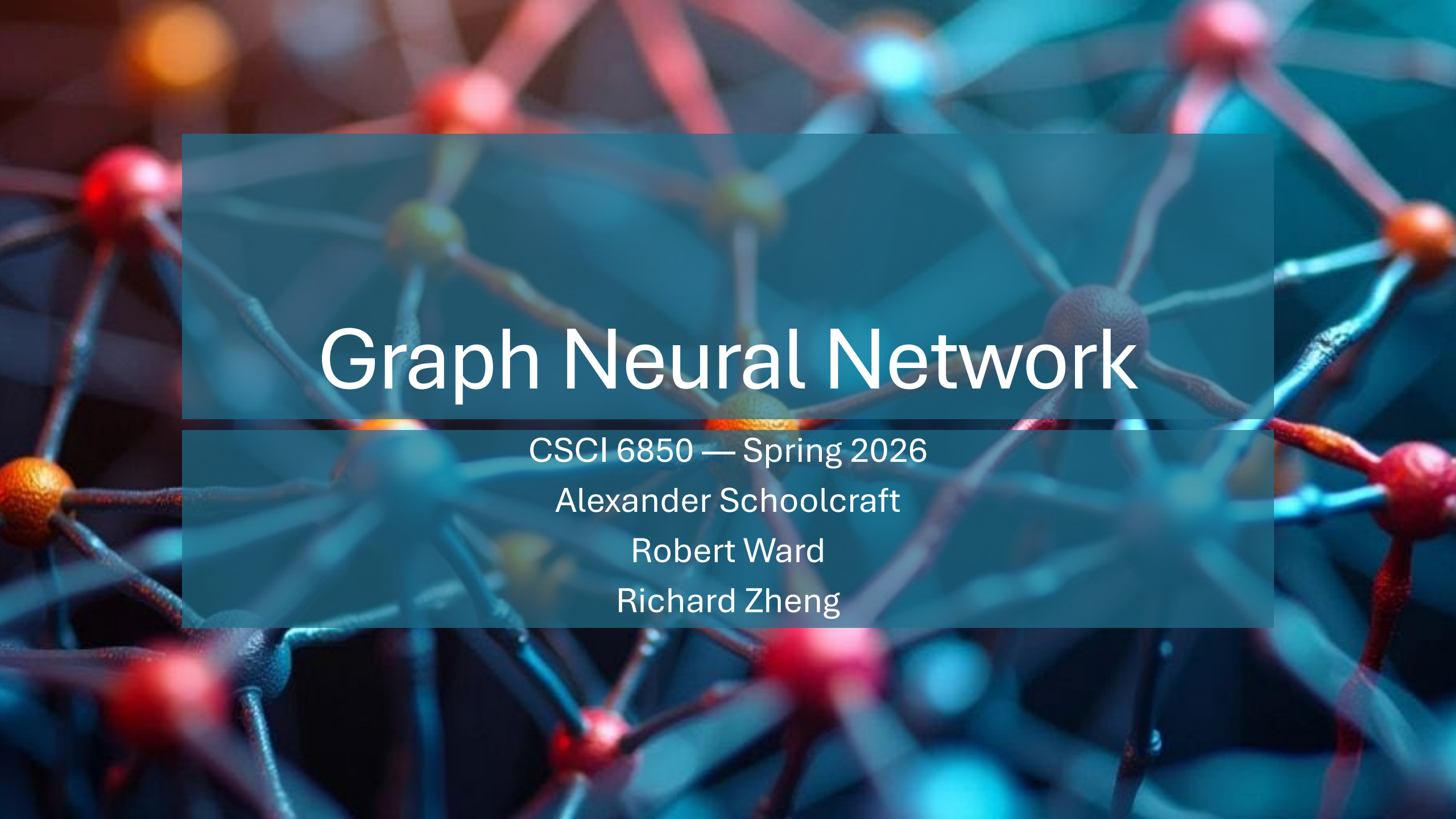




Graph Neural Network

CSCI 6850 — Spring 2026

Alexander Schoolcraft

Robert Ward

Richard Zheng

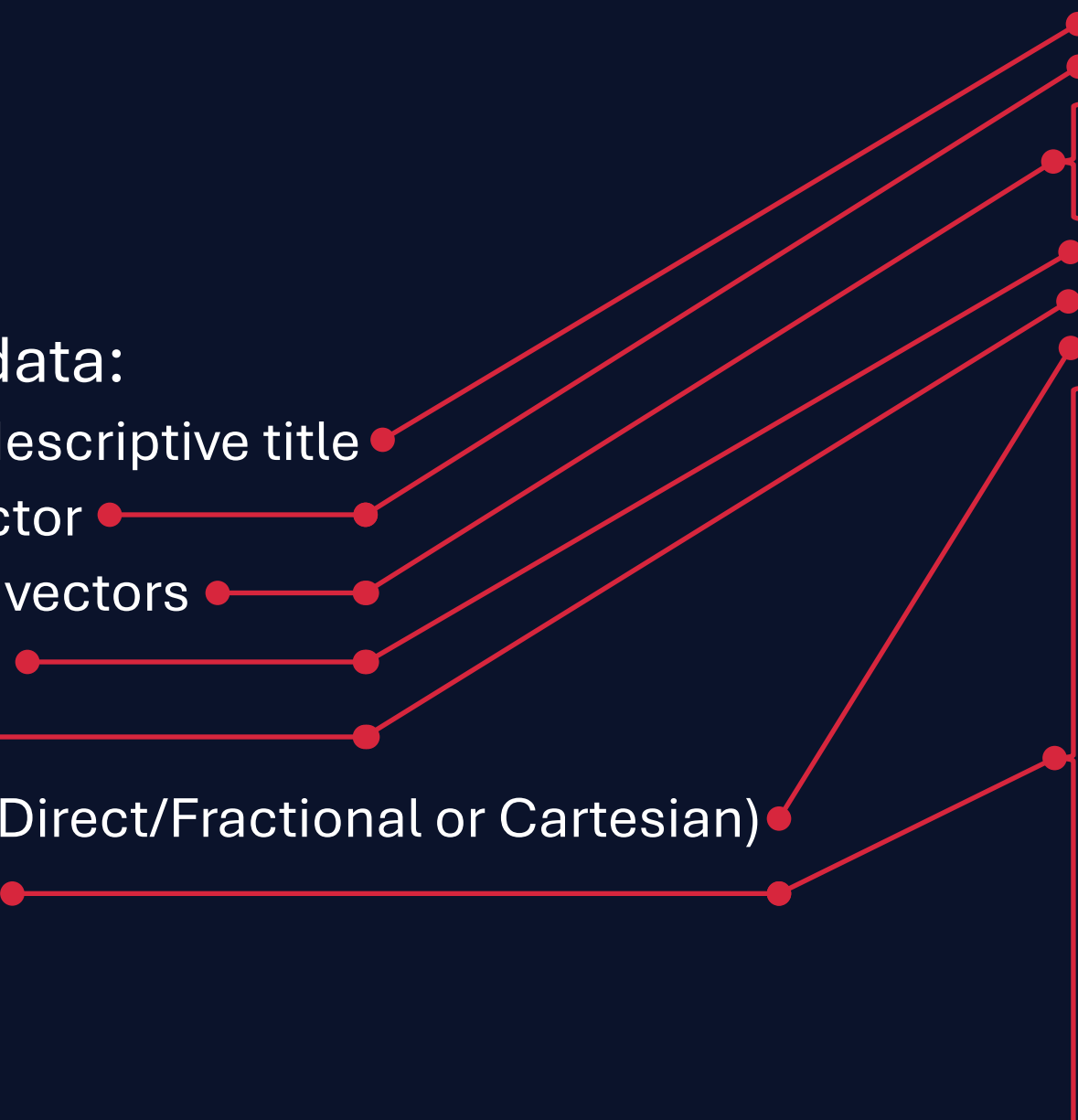


Dataset

POSCAR and Charges

POSCAR

- Plain text files
- Several points of data:
 - Comment line – descriptive title
 - Global scaling factor
 - 3x3 matrix lattice vectors
 - Element symbols
 - Atom Counts
 - Coordinate type (Direct/Fractional or Cartesian)
 - Atomic positions



```
FCC BaO
9.8
2.0 0.0 0.0
0.0 2.0 0.0
0.0 0.0 2.0
Ba O
32 32
Direct
0.0 0.0 0.0
0.0 0.25 0.25
0.25 0.0 0.25
0.25 0.25 0.0
0.5 0.0 0.0
0.5 0.25 0.25
0.75 0.0 0.25
0.75 0.25 0.0
0.0 0.5 0.0
0.0 0.75 0.25
0.25 0.5 0.25
0.25 0.75 0.0
0.5 0.5 0.0
0.5 0.75 0.25
0.75 0.5 0.25
0.75 0.75 0.0
0.0 0.0 0.5
```

Charges

- Plain text files
- List of positive and negative charges
- Each line corresponds to the same atom in the corresponding POSCAR file (e.g., CHARGE_59 corresponds to CONFIG_59)

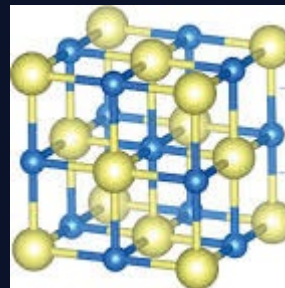
[illegible]

From Atomic Data to Graph Learning

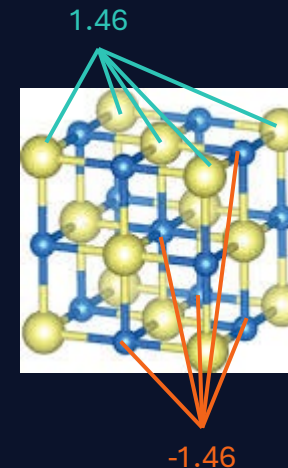
- POSCAR defines the graph structure used by the GNN
 - Atoms (Nodes)
 - Distances (Edges)
- Charge files provide per-node target values for supervised learning

```
FCC BaO
9.8
2.0 0.0 0.0
0.0 2.0 0.0
0.0 0.0 2.0
Ba O
32 32
Direct
0.0 0.0 0.0
0.0 0.25 0.25
0.25 0.0 0.25
0.25 0.25 0.0
0.5 0.0 0.0
0.5 0.25 0.25
0.75 0.0 0.25
0.75 0.25 0.0
0.0 0.5 0.0
0.0 0.75 0.25
0.25 0.5 0.25
0.25 0.75 0.0
0.5 0.5 0.0
0.5 0.75 0.25
0.75 0.5 0.25
0.75 0.75 0.0
0.0 0.0 0.5
```

Data Processing



Node-Level Regression





Graph Construction

From POSCAR to PyG dataset

Dataset Processing

- Validate dataset structure
 - Ensure PASCAR and CHARGESSS directories are present
- Discover and pair samples
 - Match CONFIG_n with corresponding CHARGE_n
- Construct feature space
 - Build vocabulary of atomic element types
- Convert raw data → graph representation
 - Load atomic structure + charge values
 - Generate nodes, edges, and features via build_graph()
- Assemble dataset for training
 - Collate graphs into PyTorch Geometric format
 - Store metadata (sample IDs, feature dimensions)
- Persist processed dataset
 - Save as .pt file for efficient reuse

```
def process(self) -> None:
    self._validate_directories()

    config_files = self._discover_config_files()
    symbol_to_idx = self._build_symbol_vocab(config_files)

    graphs: List[Data] = []
    sample_ids: List[int] = []

    for config_path in config_files:
        sample_id = extract_numeric_suffix(config_path)
        charge_path = self.charge_dir / f"CHARGE_{sample_id}"

        sample = load_sample(config_path, charge_path)
        graph = build_graph(sample, symbol_to_idx)

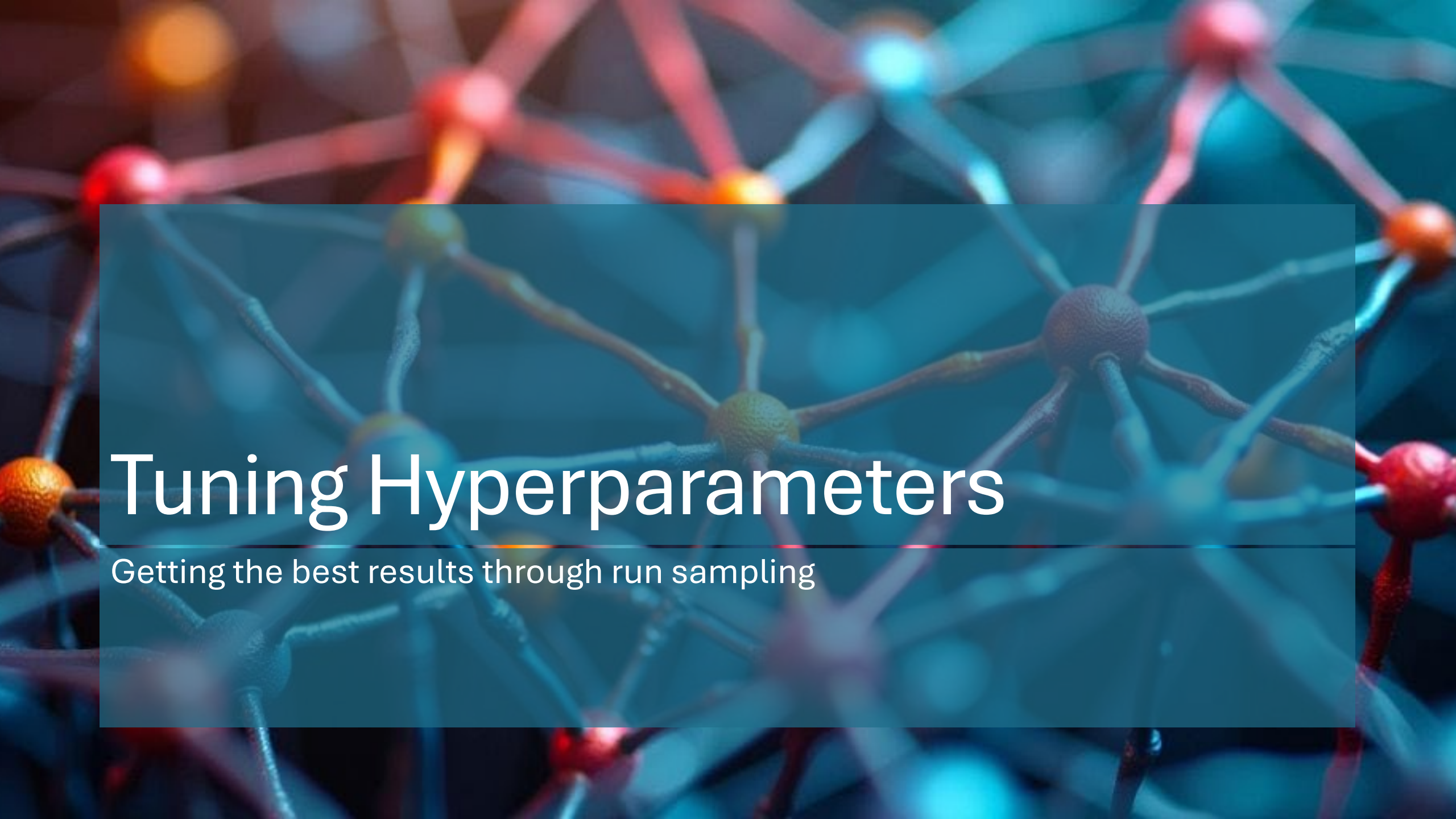
        graphs.append(graph)
        sample_ids.append(sample_id)

    if self.verbose:
        mean_degree = graph.num_edges / graph.num_nodes
        print(
            f"Processed sample {sample_id:>4d} | "
            f"nodes={graph.num_nodes:>6d} | "
            f"edges={graph.num_edges:>7d} | "
            f"mean_degree={mean_degree:>6.2f} | "
            f"cutoff={float(graph.cutoff):.2f}"
        )

    data, slices = self.collate(graphs)

    payload = {
        "data": data,
        "slices": slices,
        "symbol_to_idx": symbol_to_idx,
        "sample_ids": sample_ids,
        "num_node_features": len(symbol_to_idx),
    }

    self.processed_dir and Path(self.processed_dir).mkdir(parents=True, exist_ok=True)
    torch.save(payload, self.processed_paths[0])
```

Tuning Hyperparameters

Getting the best results through run sampling

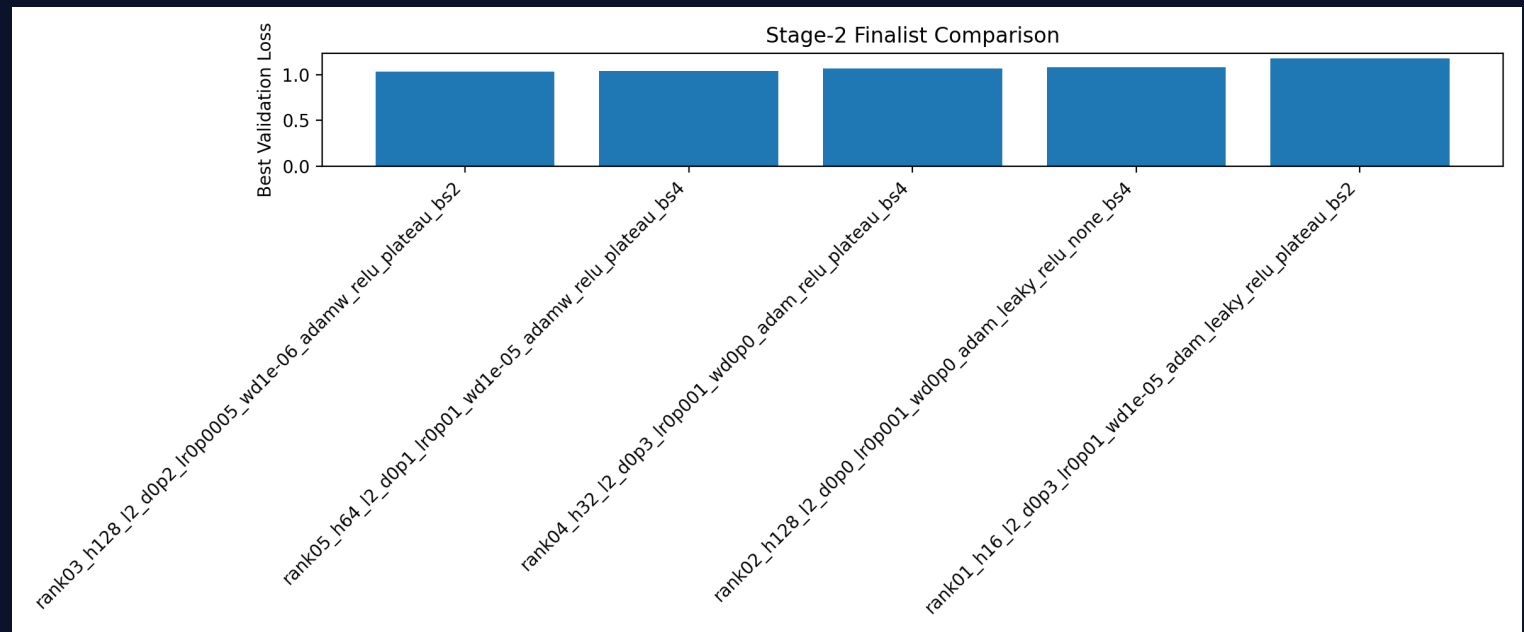
Sample Search Space

- hidden_channels
 - 16, 32, 64, 128
- num_layers
 - 2, 3, 4
- dropout
 - 0.0, 0.1, 0.2, 0.3
- lr (learning rate)
 - 1e-2, 1e-3, 5e-4, 1e-4
- weight_decay
 - 0, 1e-6, 1e-5, 1e-4
- optimizer_name
 - adam, adamw
- activation
 - relu, leaky-relu, elu
- scheduler_name
 - none, plateau
- batch_size
 - 2, 4, 8

```
def sample_search_space(max_trials: int = 20, seed: int = 42):  
    """  
    Randomly sample configurations from a structured search space.  
    """  
    random.seed(seed)  
  
    search_space = {  
        "hidden_channels": [16, 32, 64, 128],  
        "num_layers": [2, 3, 4],  
        "dropout": [0.0, 0.1, 0.2, 0.3],  
        "lr": [1e-2, 1e-3, 5e-4, 1e-4],  
        "weight_decay": [0.0, 1e-6, 1e-5, 1e-4],  
        "optimizer_name": ["adam", "adamw"],  
        "activation": ["relu", "leaky_relu", "elu"],  
        "scheduler_name": ["none", "plateau"],  
        "batch_size": [2, 4, 8],  
    }  
  
    all_configs = list(product(  
        search_space["hidden_channels"],  
        search_space["num_layers"],  
        search_space["dropout"],  
        search_space["lr"],  
        search_space["weight_decay"],  
        search_space["optimizer_name"],  
        search_space["activation"],  
        search_space["scheduler_name"],  
        search_space["batch_size"],  
    ))  
  
    random.shuffle(all_configs)  
    all_configs = all_configs[:max_trials]  
  
    configs = []  
    for cfg in all_configs:  
        configs.append({  
            "hidden_channels": cfg[0],  
            "num_layers": cfg[1],  
            "dropout": cfg[2],  
            "lr": cfg[3],  
            "weight_decay": cfg[4],  
            "optimizer_name": cfg[5],  
            "activation": cfg[6],  
            "scheduler_name": cfg[7],  
            "batch_size": cfg[8],  
        })  
  
    return configs
```

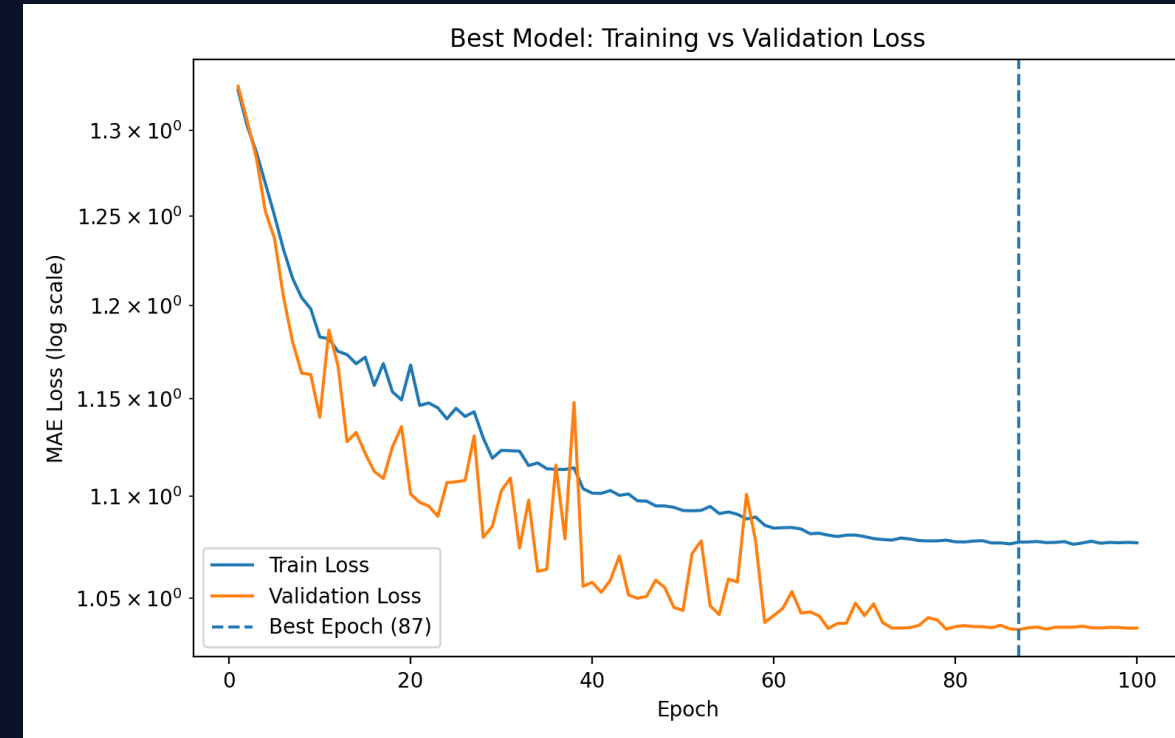
Initial test runs

- 50 different combinations used
 - Each combination ran for 5 epochs
- Recorded best validation loss for each combination
 - MAE – Mean Absolute Error
- Top 5 combinations re-ran
 - Full 100-epoch run with early stopping after a patience of 15
- Final top configuration:
 - 128 hidden channels
 - 2 layers
 - 20% dropout
 - $5e-4$ learning rate
 - $1e-6$ weight decay
 - AdamW optimizer
 - ReLU activation
 - Plateau scheduler
 - Batch Size 2
 - 17025 trainable parameters



Training Details

- Dataset was split into 70% training, 15% validation, 15% testing
- Significant performance variance across hyperparameter configurations
 - Larger models did not consistently outperform smaller models
 - Performance sensitive to learning rate, depth, and dropout
- Final configuration ran for the full 100 epochs, used epoch 87 as best results
 - Best validation loss was 1.034964 MAE
 - Early stopping halted training once validation improvements stabilized
 - Validation loss consistently tracked training loss with no severe overfitting
- Training showed rapid initial convergence, followed by plateau behavior
- Training performed on remote GPU (CUDA-enabled Tesla V100)



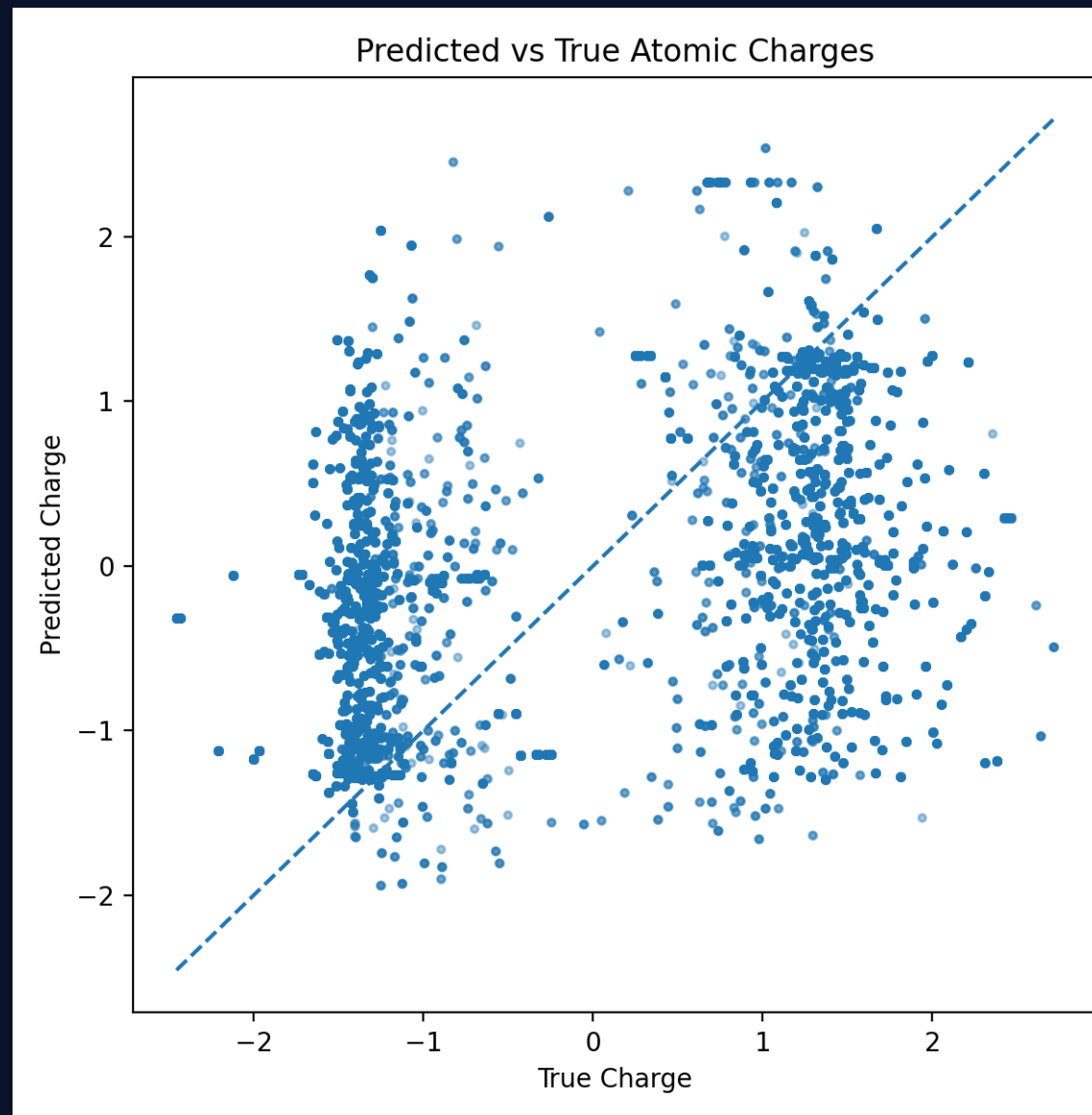
The background of the slide is an abstract, complex network of nodes and edges. The nodes are represented by small, textured spheres in various colors including red, orange, yellow, and dark blue. They are interconnected by thin, translucent lines or ribbons that form a dense, web-like structure. The overall color palette is dominated by cool blues and teals, with warmer tones (red, orange) providing contrast. The lighting is soft, creating a sense of depth and highlighting the intricate connections between the nodes.

Results

Initial Results

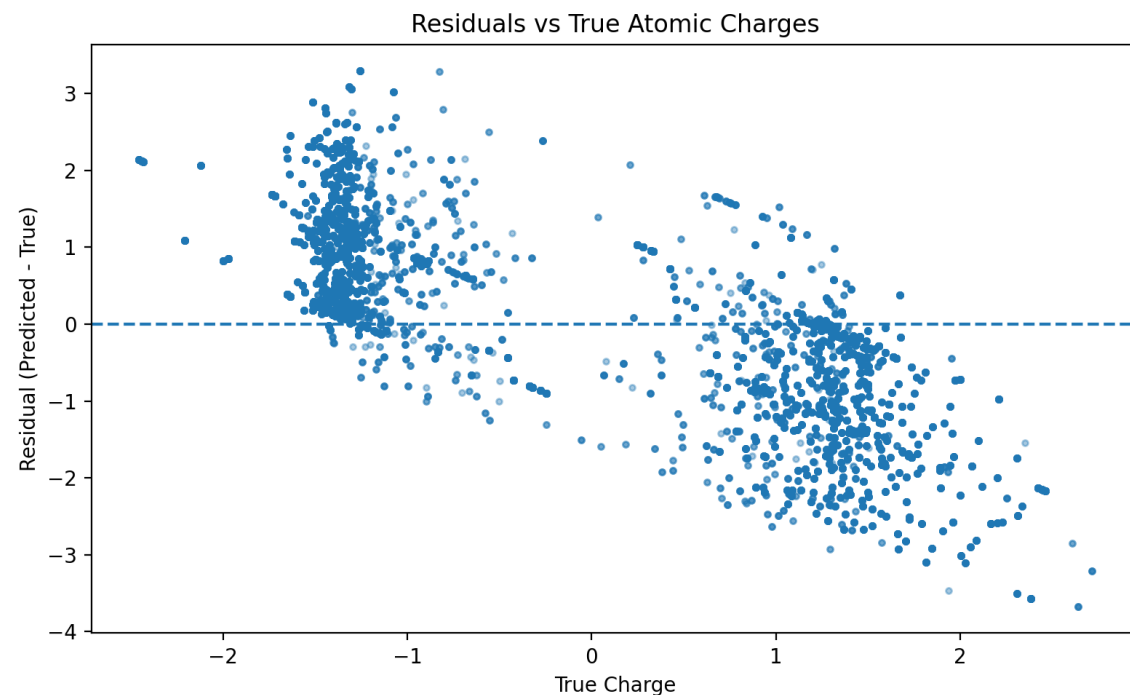
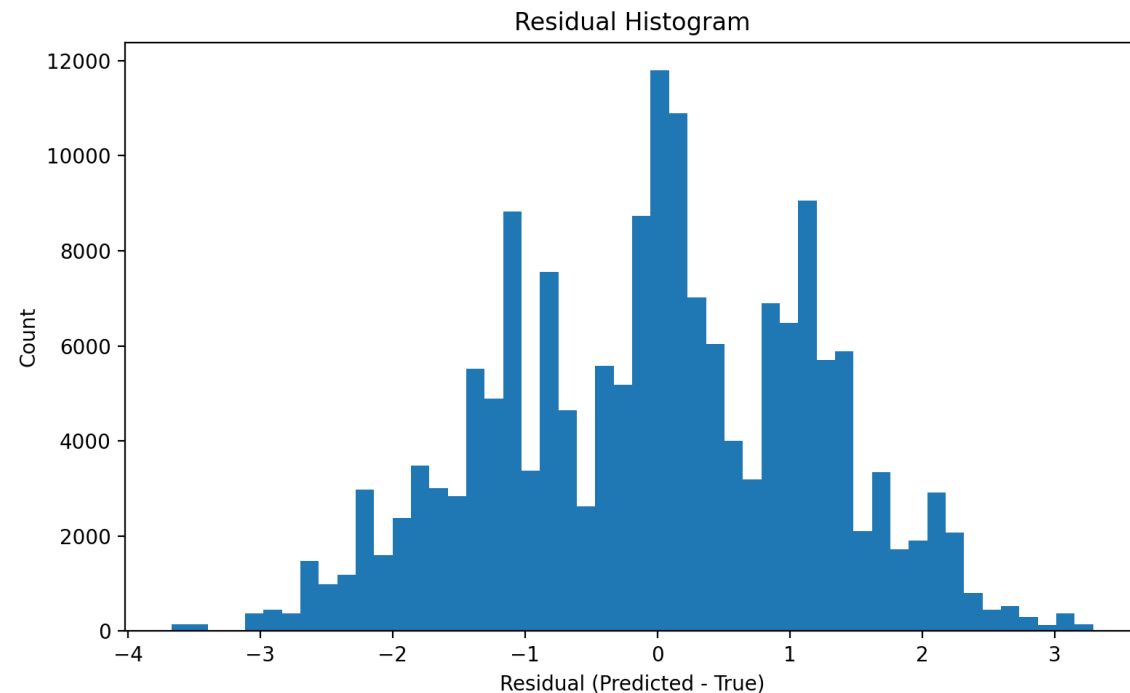
- MAE: 0.980649
 - Large relative to charge range (approx. [-2.5, 2.5])
- MSE: 1.472507
- RMSE: 1.213469
- R^2 : 0.200729
 - Only about 20% of the variance is due to the model
- Model underfits the data, failing to capture structural relationships between atoms
 - Simple GCN has limited expressiveness

The model is not just noisy — it is systematically collapsing predictions towards the mean, which indicates it is not learning node-level distinctions.



Residuals

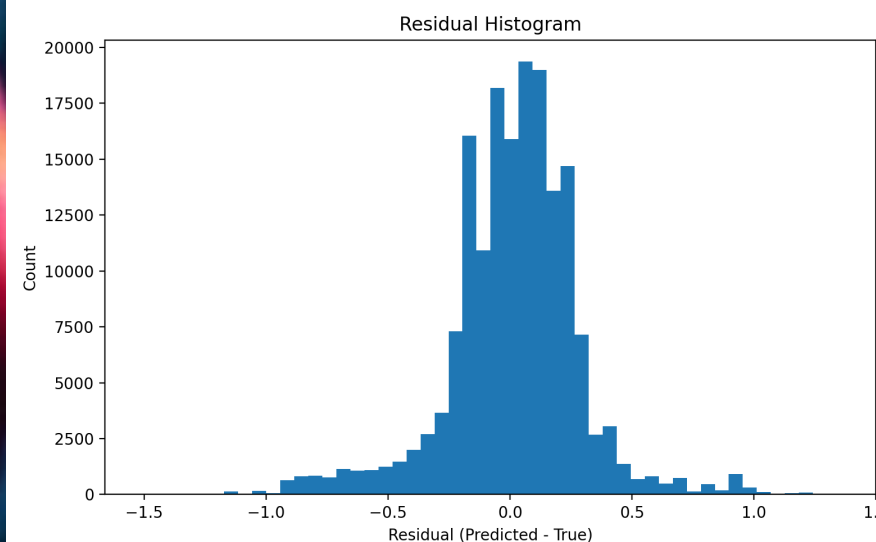
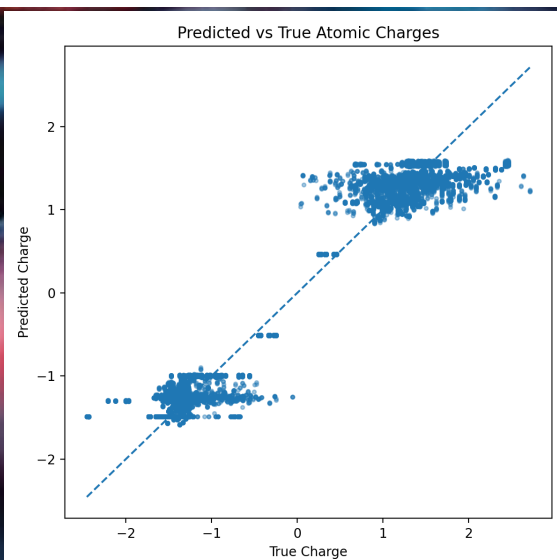
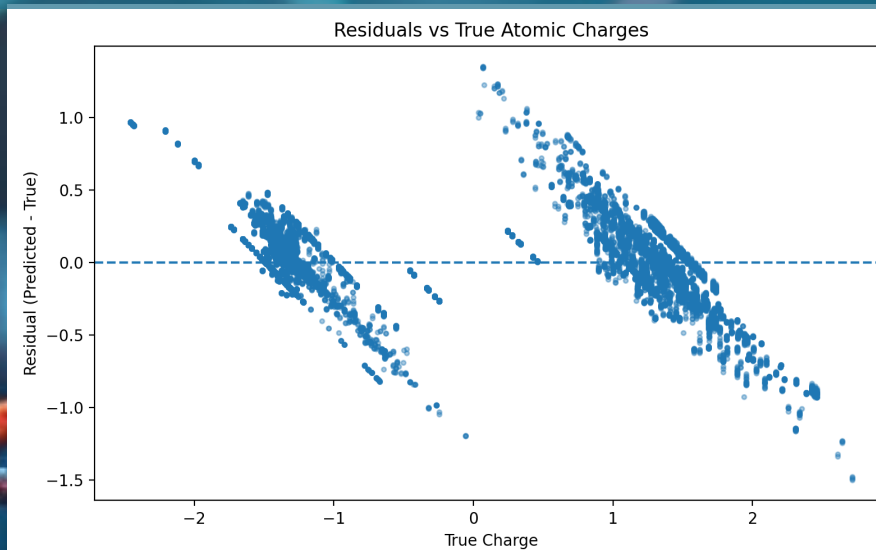
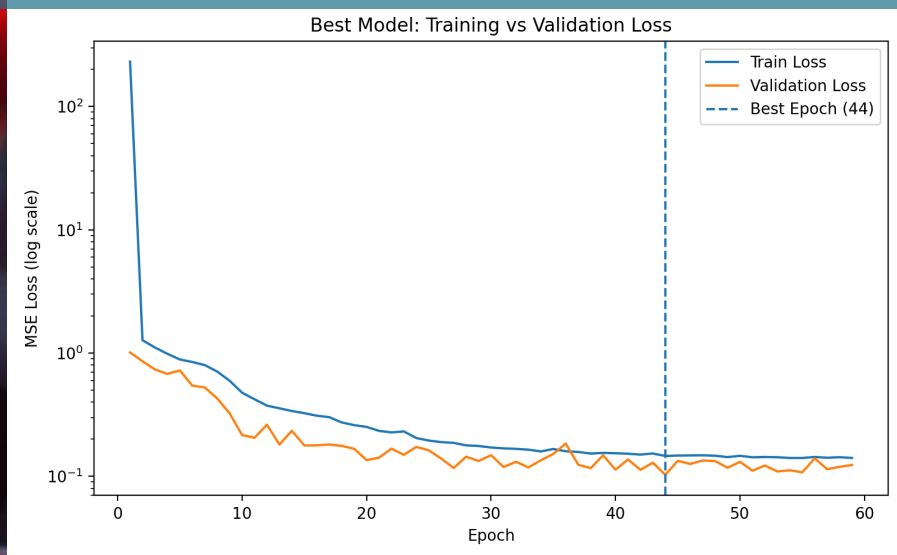
- Wide distribution
 - Residuals should ideally be tightly centered near 0
 - Wide distribution indicates poor convergence
 - Still generally a Gaussian curve, but widespread indicates poor convergence
- Residuals vs True Atomic Charges plot shows a general shift towards the median
 - True negative values tend to be over-estimated
 - True positive values tend to be under-estimated





Impact of Graph Construction on Model Performance

Adaptive (nearest-neighbor) graph preserves structure, leading to significantly improved predictions
Fixed-radius graph, while more chemically accurate, introduces excess connectivity, resulting in oversmoothing





Conclusions

Key Findings

- GNN successfully constructed and trained on atomic graph data
- Model achieved stable convergence but limited predictive performance
- Final MAE (~ 0.98) and R^2 (~ 0.20) indicate weak node-level accuracy
- Model underfits due to:
 - Limited node features
 - Simple GCN architecture
 - Loss of structural information in graph construction
- Graph construction critically impacts GNN performance
 - Fixed radius connectivity led to over-smoothing and loss of local structure



Questions?